

CARELINK — Implementation Summary & Handoff Guide

Version: 0.1.0 **Date:** 19 September 2026

CARELINK is an independently built FastAPI application for clinical care coordination. It provides a database-backed REST API for the eight requested clinical workflow areas and a lightweight browser landing page.

What has been implemented

Area	Implemented capability
Foundation	FastAPI, SQLAlchemy 2.x, SQLite configuration, safe explicit database initialization, OpenAPI documentation and UTC-aware input validation.
Patients & admission	Patients with CARELINK numbers, possible-duplicate flag, next of kin, facilities, wards, beds, users, and transactional admission/bed occupancy.
Vitals	Complete, partial and not-measured states; paired blood pressure; range validation; admission history; documented abnormal-vital alert generation.
Medication	Active medication orders and administrations, including refusal/held/not-administered reason rules.
Investigations	Multi-test requests with controlled REQUESTED → PROCESSING → AVAILABLE lifecycle and critical-result alerts.
Clinical workflows	Doctor reviews, clinical orders, structured nursing handover, alerts with acknowledge/resolve lifecycle, and transactional discharge.
Auditability	Audit records for important creates, transitions, alert actions, and discharge; the calling service owns the transaction.

Project layout

app/main.py	Application and health endpoint
app/core/config.py	Environment configuration
app/database/	Connection, models and explicit init command
app/workflows/	Schemas, services and REST routes
frontend/index.html	Browser landing workspace
tests/	Automated workflow tests

Run the application

1. Create and activate a Python 3.12+ virtual environment.
2. Install: `pip install -e '[dev]'`.
3. Copy `.env.example` to `.env` if needed.
4. Create tables: `python -m app.database.init_db`.
5. Start: `uvicorn app.main:app --reload`.

Use `/docs` for interactive API documentation, `/app` for the browser workspace and `/health` for service status.

Recommended API workflow

1. Create a user; use role DOCTOR for prescribing, reviews and discharge.
2. Create a facility, ward and bed.
3. Create a patient and optionally add next of kin.
4. Create an admission. This occupies the selected available bed.
5. Record vitals, medications, investigations, reviews, handovers and alerts against the admission.
6. Move each investigation item to processing before entering its result.
7. Acknowledge or resolve relevant alerts.
8. Discharge using a doctor user. This returns the bed to AVAILABLE atomically.

Important business rules

- The bed must belong to the selected ward and be AVAILABLE before admission.
- Vital and medication-administration timestamps require a timezone and are normalized to UTC.
- COMPLETE vitals require all values; PARTIAL requires some values plus a reason; NOT_MEASURED requires no values and a reason.
- Administration is permitted only for ACTIVE medication orders. Reasons are required for NOT_ADMINISTERED, REFUSED and HELD.
- Results cannot become available before an investigation item is processing.
- SpO₂ below 90% or systolic BP at least 180 creates a HIGH abnormal-vital alert.
- Only an ACTIVE admission may be discharged; discharge requires an active DOCTOR user.

Verification performed

Three automated tests passed. They cover health status, admission-to-discharge bed lifecycle, abnormal-vital alert generation and invalid vital validation. OpenAPI generation was checked and exposes 28 paths.

Production follow-up

Important: Passwords use a one-way script hash, but the current build does not yet implement login/JWT authentication or request-bound role authorization. Add these, migrations, a richer frontend, access controls,

logging/monitoring, encryption/key management, backups and formal clinical governance review before production use.